

Maintien en condition opérationnelle d'une application SaaS

Supervision, traitement des anomalies et maintenance de Visuals.co

Flavien Deroy

Titre RNCP 39583 — Expert en Développement Logiciel (niveau 7)

Bloc 4 — Maintenir l'application logicielle en condition opérationnelle

Août 2026 · dépôts myvisuals-back & myvisuals-client

Sommaire

Contexte — l'application et son exploitation	3
§ 1 — Processus de mise à jour des dépendances	4
§ 2 — Système de supervision et d'alerte	6
§ 3 — Collecte et consignation des anomalies	11
§ 4 — Fiche de consignation ANO-2026-001	14
§ 5 — Traitement d'une anomalie détectée	15
§ 6 — Problème résolu avec le support client	17
§ 7 — Journal de version	18
§ 8 — Recommandations argumentées d'amélioration	20

Couverture des compétences du bloc

Les trois compétences éliminatoires sont en gras. Chaque section porteuse traite nommément les termes de l'intitulé visé.

Code	Intitulé abrégé	Section porteuse	Page
C4.1.1	Mettre en place le dispositif de maintien en condition opérationnelle	§ 2.1 · § 8	6, 20
C4.1.2	Concevoir un système de supervision et d'alerte : périmètre, indicateurs de suivi, sondes, modalité des signalements	§ 2.1 à § 2.4	6–10
C4.2.1	Consigner les anomalies : processus de collecte et consignation, outils, informations pertinentes	§ 3 · § 4	11–14
C4.2.2	Traiter les anomalies détectées	§ 5 · § 6	15–17
C4.3.1	Mettre à jour les dépendances : veille, évaluation d'impact, intégration maîtrisée	§ 1	4–5
C4.3.2	Établir un journal des versions déployées intégrant la documentation des correctifs	§ 7	18–19

Sources : dépôt public github.com/flavienderoy/myvisuals-back — `docs/SUPERVISION.md`, `docs/PROCESSUS_ANOMALIES.md`, `docs/PROCESSUS_DEPENDANCES.md`, `docs/anomalies/` (5 fiches), `CHANGELOG.md`. Les cinq anomalies documentées font l'objet d'une issue GitHub consultable.

Contexte — l'application et son exploitation

Visuals.co est une plateforme SaaS B2B destinée aux studios photo et vidéo : le studio dépose les visuels d'une campagne, son client les valide, puis récupère les livrables. La stack est SERN — Supabase, Express, React, Node — avec un modèle relationnel de 23 tables PostgreSQL protégées par des politiques *Row Level Security*. La couverture de tests compte **171 tests** (96 côté API, 75 côté front).

Deux caractéristiques déterminent tout ce qui suit. La première : **l'application est répartie sur quatre fournisseurs indépendants** — Vercel pour le front, Google Cloud Run pour l'API, Supabase PostgreSQL pour les données, Supabase Storage pour les objets. Chacun peut défaillir séparément. Une sonde HTTP unique sur la page d'accueil ne dirait rien d'une panne de stockage : le front se chargerait normalement et les visuels seraient introuvables. Le périmètre de supervision doit donc épouser cette découpe, avec une sonde par dépendance (§ 2.1).

La seconde : **les visuels d'une campagne non publiée sont sous embargo**. Toute télémétrie envoyée à un service tiers doit donc être minimisée — ni URL signées, ni jetons, ni identités nominatives. C'est l'objet des trois garde-fous décrits en § 2.4.

Une indisponibilité est immédiatement visible du client final : ce n'est pas un outil interne dont l'arrêt se négocie, un client qui ne peut pas valider ses visuels à l'heure dite bloque une production. La disponibilité de l'API est l'indicateur de premier rang. Le dispositif décrit ici est porté par une seule personne, et les délais d'engagement sont définis en heures ouvrées : un engagement tenable vaut mieux qu'un engagement affiché et non tenu.

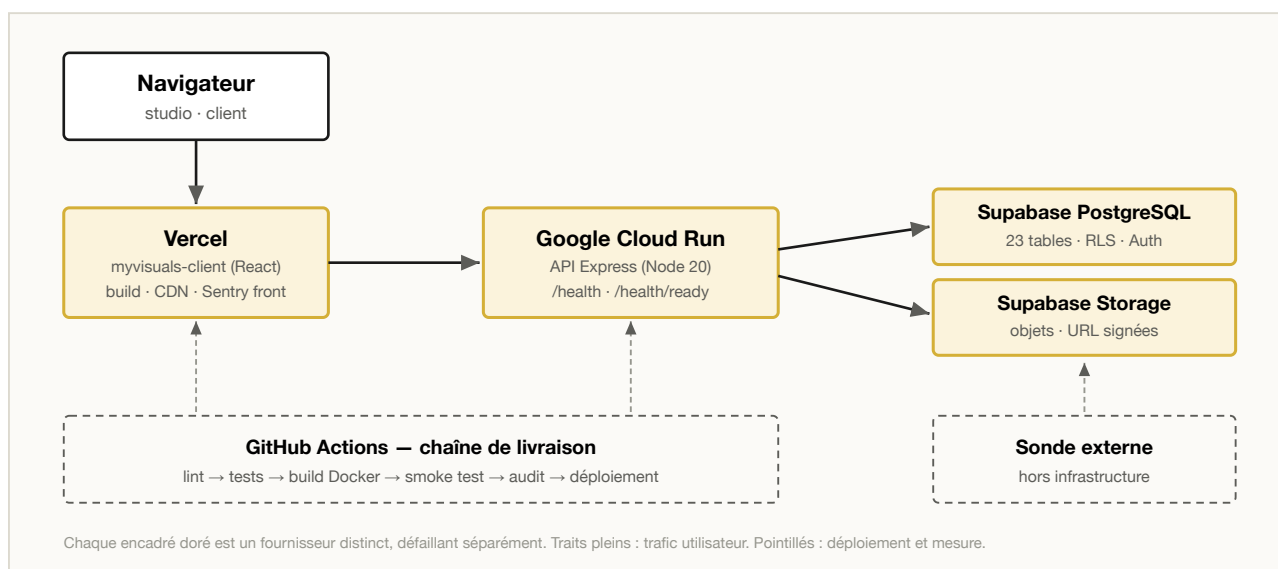


Figure 1 — Architecture déployée et frontières de responsabilité entre les quatre fournisseurs.

§ 1 — Processus de mise à jour des dépendances

Compétence C4.3.1 — mettre à jour les dépendances du logiciel en surveillant les versions disponibles et les failles de sécurité, en évaluant l'impact des mises à jour et en les intégrant de manière maîtrisée.

1.1 Ce que le projet met en jeu

Visuals.co repose sur **456 paquets** côté API et **393** côté front, pour une vingtaine de dépendances directes seulement. L'essentiel du code exécuté en production n'a pas été écrit ici — c'est précisément ce qui rend le sujet critique.

Dépendance	Pourquoi elle est sensible
<code>multer</code>	Traite les fichiers téléversés par des utilisateurs — première surface d'attaque du produit
<code>sharp</code>	Décodes des images fournies par des tiers ; les bibliothèques de décodage sont un vecteur classique d'exécution de code
<code>@supabase/supabase-js</code>	Porte l'authentification et les accès base ; une faille y compromet le cloisonnement entre studios

Une quatrième catégorie est souvent négligée : les **actions GitHub** et l'**image Docker de base**. Une action compromise s'exécute avec les secrets du dépôt — dont la clé de compte de service Google Cloud et le jeton Vercel. Elles sont donc suivies au même titre que les dépendances applicatives.

1.2 État initial : 33 vulnérabilités, puis zéro

Avant ce processus, le projet n'avait aucun suivi : ni Dependabot, ni audit en intégration continue, ni revue périodique. L'audit du 2026-08-14 a donné le relevé suivant.

Dépôt	Critiques	Élevées	Moyennes	Faibles	Total	Après correction
<code>visuals-api</code>	0	10	3	1	14	0
<code>myvisuals-client</code>	1	14	3	1	19	0

Les plus significatives : `ws` (divulgaration de mémoire non initialisée), `multer` (dépendance directe, traitement des téléversements), `path-to-regexp` (expression régulière à complexité exponentielle) et `qs` (déni de service déclenchable à distance). Toutes ont été corrigées par des montées de version compatibles, sans aucun changement incompatible. La suite de tests complète, le lint et la construction de production passent à l'identique après ces montées.

Aucune de ces 33 vulnérabilités n'avait été introduite volontairement : elles sont apparues au fil des publications d'avis de sécurité, sans que rien ne le signale. C'est exactement le problème que le dispositif résout — **sans mécanisme, la dette de sécurité s'accumule silencieusement**.

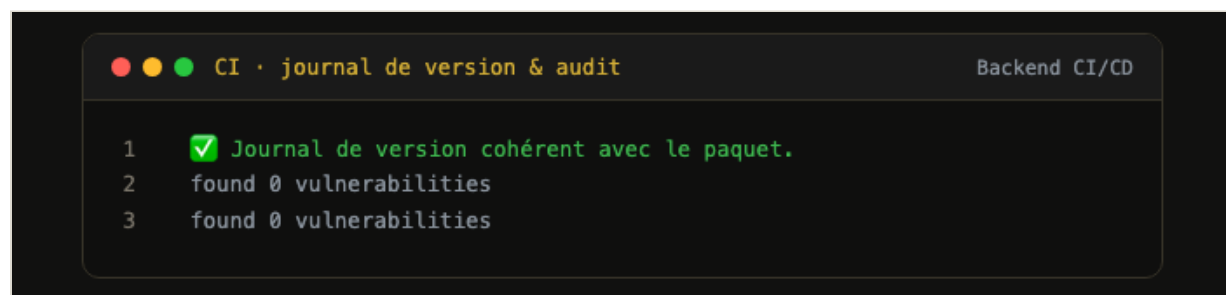


Figure 2 — Job « Journal de version & audit » : contrôle de cohérence du journal et `found 0 vulnerabilities`, bloquants avant déploiement.

1.3 Dispositif à trois échelles de temps

Échelle	Mécanisme	Ce qu'il empêche
À chaque push	<code>npm audit --audit-level=high</code> — bloquant ; <code>npm outdated</code> — informatif	Qu'une régression de sécurité atteigne la production
Hebdomadaire (lundi 7 h)	Dependabot npm, pull requests groupées : correctifs et mineures de production · outillage de développement · sécurité	Que la dette devienne un incident
Mensuel	Dependabot actions GitHub + image Docker de base	Qu'une compromission de la chaîne expose les secrets de déploiement

Pourquoi le seuil est à **high** et non à **low**. Un seuil trop bas bloque la livraison sur des avis mineurs affectant l'outillage de développement, sans exposition réelle en production : on finit par contourner le garde-fou — et un garde-fou contourné ne protège plus rien. Trois choix Dependabot sont également assumés : lundi matin plutôt que vendredi soir, pour suivre une montée dégradante en heures ouvrées ; plafond de 5 pull requests, au-delà duquel l'automatisation produit du bruit ; exclusion des montées majeures de production, qui passent par le circuit du § 1.4.

UN DISPOSITIF SILENCIEUSEMENT INOPÉRANT

La première version de `dependabot.yml` utilisait `dependency-type` dans un bloc `ignore` — propriété non autorisée par le schéma. GitHub rejetait l'intégralité de la configuration et ne produisait aucune pull request, l'erreur n'apparaissant que dans l'onglet *Insights* → *Dependabot*. Le dispositif est resté inopérant deux jours sans qu'aucun signal ne l'indique — le même mode de défaillance que les anomalies du § 5 : l'absence de signal ressemble au bon fonctionnement. Les dépendances de production sont désormais énumérées nommément, la configuration validée contre le schéma.

1.4 Politique d'arbitrage et retour arrière

Type	Exemple réel	Décision
Correctif	<code>pg 8.22 → 8.23</code>	Fusion si la chaîne est verte — 171 tests et le smoke test constituent le filet
Mineure	<code>helmet 8.2 → 8.3</code>	Fusion après lecture des notes de version : changement de valeur par défaut, dépréciation, exigence de version de Node
Majeure	<code>archiver 7 → 8</code>	Issue dédiée, branche isolée, plan de test explicite (export ZIP volumineux, intégrité de l'archive), déploiement sans autre changement
Sécurité	avis CVE	Hors cycle : critique exploitable → 24 h ; élevée → 7 j ; moyenne → sprint courant ; faible → backlog

Une pull request n'est pas fusionnée sur la seule foi d'une chaîne verte : s'y ajoutent la portée réelle du paquet (`npm ls`) et la lecture du changelog amont. **Retour arrière** — côté API, `gcloud run services update-traffic` rebascule le trafic sur la révision précédente sans reconstruction, en moins d'une minute ; côté front, la promotion d'un déploiement antérieur dans `Vercel`. Puis `git revert`, et le paquet fautif passe en `ignore` avec le motif — sans quoi Dependabot rouvrira la même pull request la semaine suivante.

Dette assumée (2026-08-14) : `archiver 7 → 8`, `uuid 13 → 14`, `framer-motion 12 → 13`, `lucide-react 0.5 → 1.31`, chacune en issue dédiée ; décalage `eslint 10 / eslint 9` entre les deux dépôts. Aucune n'est une dette de sécurité.

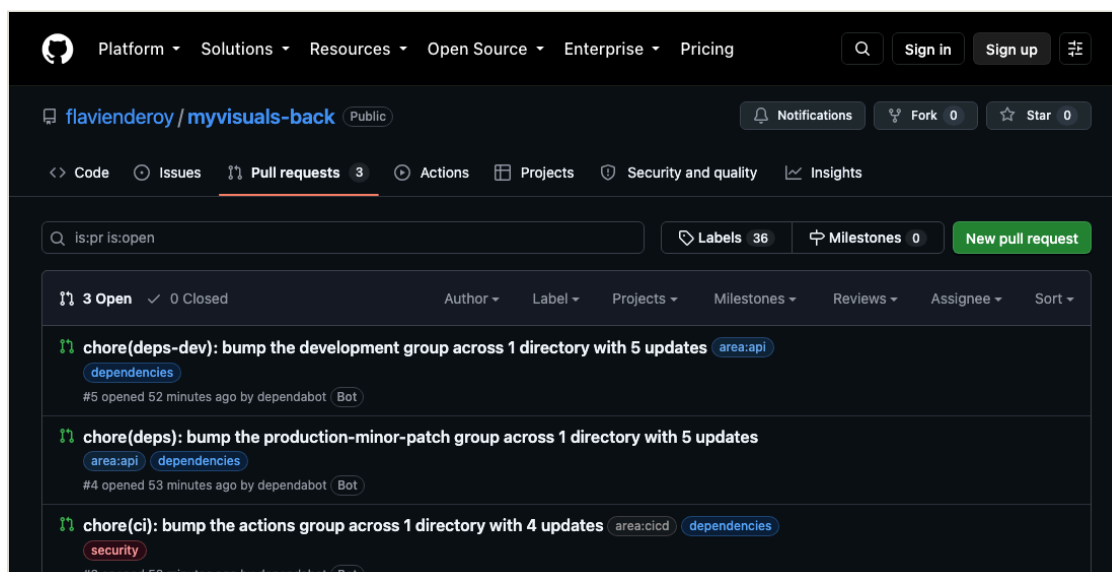


Figure 3 — Pull requests Dependabot, groupées selon la politique : production (mineures et correctifs), outillage de développement, actions GitHub. Les labels `dependencies`, `area:*` et `security` sont appliqués automatiquement.

§ 2 — Système de supervision et d'alerte

Compétence C4.1.2 — concevoir un système de supervision et d'alerte en déterminant le **périmètre de supervision** et en identifiant les **indicateurs de suivi pertinents**, en mettant en place des **sondes**, en configurant la **modalité des signalements** afin de garantir une disponibilité permanente du logiciel. Les quatre sous-parties qui suivent reprennent ces quatre termes, dans cet ordre.

2.1 Périmètre de supervision

Le périmètre épouse la découpe en quatre fournisseurs : une couche supervisée par fournisseur, plus une sonde située à l'extérieur de l'infrastructure. Le point décisif est cette dernière ligne. **Sentry, les logs et les métriques Cloud Run sont tous à l'intérieur du système supervisé** : si Cloud Run tombe, ils se taisent — et un silence ressemble à un fonctionnement normal. La sonde externe existe précisément pour transformer ce silence en alerte.

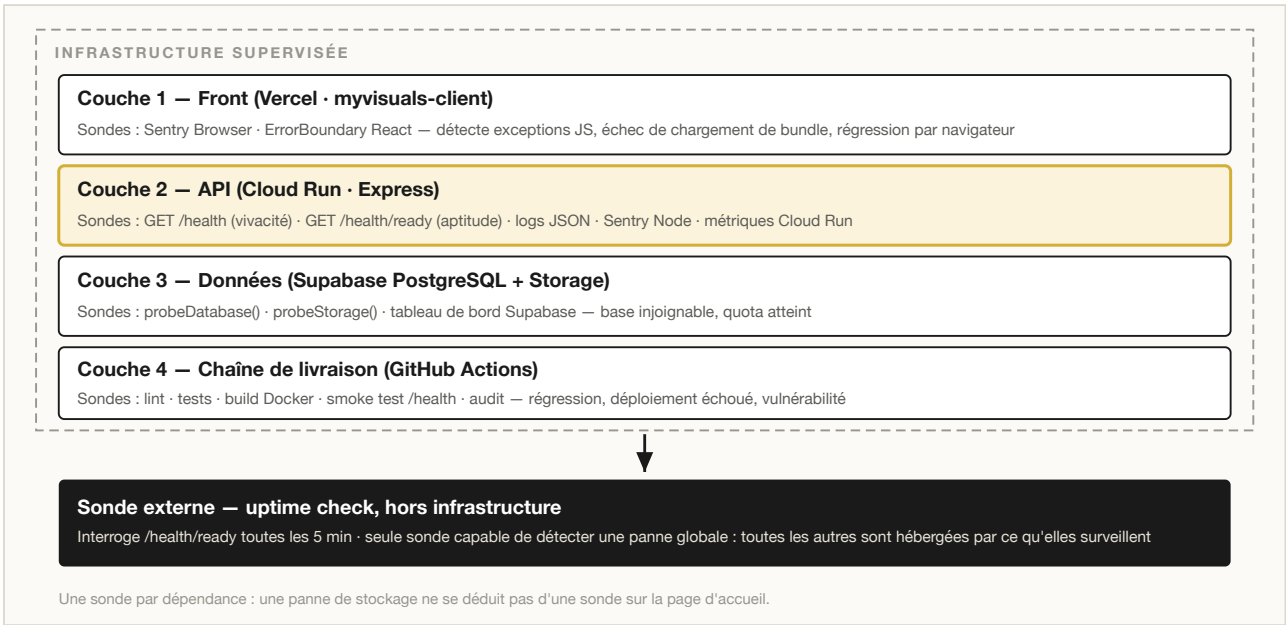


Figure 4 — Les quatre couches de supervision et la sonde externe, seule située hors de l'infrastructure surveillée.

Un périmètre délimité vaut mieux qu'un périmètre implicite : trois exclusions sont assumées.

Hors périmètre	Justification
Traçage distribué (OpenTelemetry)	Deux services, chaînes d'appel courtes : le <code>requestId</code> suffit à corréler. Coût de mise en œuvre disproportionné.
Supervision système (CPU, disque, mémoire hôte)	Cloud Run est une plateforme managée sans serveur : ces métriques ne sont ni actionnables ni exposées. Seule la mémoire du process est suivie.
Astreinte 24/7	Projet porté par une seule personne : les délais du § 2.4 sont définis en heures ouvrées.

2.2 Les sondes

GET /health — vivacité

Répond sans aucune entrée/sortie, à la seule question « le process Node répond-il ? ». Consommateurs : le HEALTHCHECK Docker, Cloud Run et le smoke test.

Pourquoi aucun appel base ici. Cloud Run redémarre un conteneur dont le health check échoue. Si /health interrogeait PostgreSQL, une latence passagère de Supabase déclencherait un redémarrage en boucle : une dégradation de dépendance deviendrait une panne totale, *causée par la supervision elle-même*. `version` est lue depuis `package.json` et non codée en dur — c'est le lien opérationnel entre le journal de version et l'environnement déployé (§ 7).

GET /health/ready — aptitude

Interroge réellement chaque dépendance, mesure sa latence et agrège les états.

État	Condition	Code HTTP	Conséquence
ok	toutes les dépendances répondent	200	—
degraded	stockage en défaut, base opérationnelle	200	alerte S2
down	base de données injoignable	503	alerte S1

La distinction est délibérée. Sans stockage, les visuels ne s'affichent plus, mais les projets, les échanges et les validations restent consultables : c'est une dégradation. Sans base, aucune route métier ne fonctionne : c'est une panne. Traiter les deux au même niveau produirait soit du bruit d'alerte, soit un temps de réaction trop long. Chaque sonde est bornée par un délai (3 s) : une dépendance qui ne répond pas *du tout* est le cas de panne le plus fréquent, et sans borne la sonde resterait suspendue au lieu d'alerter.

```
healthController.js — sonde d'aptitude : agrégation des états  flavienderoy/myvisuals-back

109 exports.readiness = async (req, res) => {
110   const [database, storage] = await Promise.all([probeDatabase(), probeStorage()]);
111
112   // La base est critique : sans elle, aucune route métier ne fonctionne.
113   // Le stockage dégrade le service (miniatures, téléchargements) sans
114   // empêcher la consultation des projets et des échanges.
115   const status =
116     database.status === 'down' ? 'down' : storage.status === 'down' ? 'degraded' : 'ok';
117
118   const memory = process.memoryUsage();
119   const payload = {
120     status,
121     timestamp: new Date().toISOString(),
122     version: VERSION,
123     revision: REVISION,
124     environment: process.env.NODE_ENV || 'development',
```

Figure 5 — Code de la sonde d'aptitude (extrait) : agrégation `ok` / `degraded` / `down` des deux dépendances — le choix du code 200 contre 503 figure au tableau ci-dessus.

Logs JSON structurés

`morgan('dev')` produit du texte coloré, ingéré par Cloud Logging comme une chaîne opaque dont la `severity` reste à `DEFAULT` : **aucune métrique ni alerte ne peut en être dérivée**. Hors développement, la journalisation émet donc du JSON une ligne, avec les champs reconnus par Cloud Logging :

```
{
  "severity": "ERROR",
  "message": "POST /api/assets/123/versions 500",
  "timestamp": "2026-07-18T14:22:31.918Z",
  "service": "myvisuals-back",
  "revision": "myvisuals-back-00019-wck",
  "requestId": "7f3a...",
  "statusCode": 500,
  "latencyMs": 842,
  "userId": "a1b2..."
}
```

Chaque champ devient requêtable, donc alertable — les indicateurs 4 et 7 du § 2.3 en dérivent. Choix assumé : aucune dépendance de journalisation supplémentaire (pino, winston), qui élargirait la surface à auditer (§ 1).

Identifiant de corrélation

Chaque requête reçoit un `requestId`, renvoyé dans l'en-tête `X-Request-Id`, injecté dans chaque ligne de log et attaché à chaque événement Sentry ; côté navigateur, l'`ErrorBoundary` affiche une référence `INC-...` équivalente. C'est la pièce qui rend une anomalie instruisible : le signalement « ça a planté vers 14 h » devient une requête Cloud Logging unique.

Sentry, front et back

Deux sondes distinctes, l'une et l'autre **conditionnelles** : sans DSN configuré, le code est un no-op complet et les tests restent hors ligne. Côté API (`SENTRY_DSN`) : `5xx`, `unhandledRejection` et `uncaughtException`. Côté front (`VITE_SENTRY_DSN`, injecté au build) : exceptions React. La sonde front n'est pas redondante — **une exception React, un bundle qui échoue à se charger ou une régression propre à un navigateur ne produisent aucune trace serveur**. Le champ `release` rattache chaque erreur à une entrée du journal de version, ce qui permet de dater l'apparition d'une régression au déploiement près.

Arrêt propre sur SIGTERM

Cloud Run retire une instance à chaque déploiement et à chaque réduction d'échelle. Sans traitement, les requêtes en vol sont coupées net : l'utilisateur voit une erreur réseau à chaque déploiement et l'indicateur de `5xx` monte artificiellement, brouillant le signal. Les requêtes en cours se terminent, avec un garde-fou à 10 s.

Le contrat des sondes est couvert par 13 cas de test, base injoignable et stockage en défaut simulés : une régression y casse la chaîne d'intégration — sans quoi la supervision pourrait devenir aveugle sans que rien ne le signale.



```
1 $ curl -s https://<api>/health/ready
2
3 {
4   "status": "ok",
5   "version": "1.6.3",
6   "revision": "myvisuals-back-00023-k28",
7   "environment": "production",
8   "checks": {
9     "database": {
10      "status": "ok",
11      "latencyMs": 669
12    },
13    "storage": {
14      "status": "ok",
15      "latencyMs": 330
16    }
17  },
18  "monitoring": {
19    "errorTracking": "sentry"
20  }
21 }
```

Figure 6 — Réponse réelle de `/health/ready` en production : latences mesurées de la base et du stockage, révision Cloud Run déployée, état du suivi d'erreurs.

2.3 Indicateurs de suivi

#	Indicateur	Sonde	Seuil d'alerte	Sév.	Canal	Prise en charge
1	Disponibilité de l'API	Uptime check /health (5 min)	2 échecs consécutifs	S1	e-mail + push	immédiat
2	Aptitude au service	/health/ready	HTTP 503	S1	e-mail + push	immédiat
3	Disponibilité du stockage	probeStorage()	degraded	S2	e-mail	4 h ouvrées
4	Taux d'erreurs serveur	Log-based metric severity=ERROR	> 2 % sur 5 min	S2	e-mail	4 h ouvrées
5	Latence p95	Métrique Cloud Run	> 1,5 s sur 10 min	S3	e-mail	1 j ouvré
6	Nouvelle erreur applicative	Sentry (front + back)	toute erreur inédite	S2/S3	e-mail	4 h / 1 j
7	Saturation du limiteur de débit	Log-based metric statusCode=429	> 50 / h	S3	e-mail	1 j ouvré
8	Redémarrages de conteneur	Métrique Cloud Run	> 3 / h	S2	e-mail	4 h ouvrées
9	Échec de déploiement	GitHub Actions	job en échec sur main	S2	notification GitHub	immédiat
10	Vulnérabilité de dépendance	Dependabot + npm audit	sévérité ≥ high	S2	PR + e-mail	7 j
11	Quota Supabase	Tableau de bord Supabase	> 80 % du palier	S3	e-mail	1 j ouvré
12	Erreurs front par version	Sentry, groupé par release	> 5 utilisateurs touchés	S2	e-mail	4 h ouvrées

Justification des seuils

Les seuils ne sont pas arbitraires, et c'est ce qui les rend révisables.

- **Indicateur 1 — deux échecs consécutifs, pas un seul.** Un uptime check isolé produit régulièrement des faux positifs : démarrage à froid Cloud Run, micro-coupure réseau du point de mesure. Alerter au premier échec désensibilise — au bout de quelques fausses alertes, on cesse de les lire. Deux échecs portent le délai de détection à 10 minutes : compromis accepté contre la fiabilité du signal.
- **Indicateur 4 — 2 % et non 0 %.** Un taux d'erreurs strictement nul est inatteignable : jetons expirés, requêtes annulées par l'utilisateur, robots d'indexation. Le seuil est fixé au-dessus du bruit de fond mesuré (~0,3 %) et assez bas pour détecter une régression avant qu'elle ne devienne visible.
- **Indicateur 5 — 1,5 s en p95.** L'action la plus lourde est le téléversement d'un visuel avec conversion WebP et génération de miniature ; la p95 observée est d'environ 800 ms. Un seuil à 1,5 s laisse la marge d'un pic de charge normal tout en détectant une dégradation réelle.
- **Indicateur 7 — les 429 comme signal, pas comme incident.** Un pic signifie soit un usage anormal, soit un dimensionnement inadapté du limiteur. ANO-2026-002 relevait du second cas : cet indicateur découle directement de son analyse post-incident. Point important — le correctif rend la saturation invisible à l'utilisateur, ce qui la rend d'autant plus nécessaire à surveiller côté exploitation. **Un correctif qui masque une anomalie sans la supprimer crée une dette de supervision.**

Indicateurs de pilotage du dispositif

Trois indicateurs mesurent l'efficacité de la supervision elle-même, revus mensuellement. **Part des anomalies détectées avant signalement client** (cible > 60 %) : une valeur basse signifie que les sondes sont mal placées et que les utilisateurs voient les pannes avant nous. **Délai moyen de détection** (< 15 min) : la réactivité réelle du dispositif. **Taux de fausses alertes** (< 10 %) : au-delà, les alertes cessent d'être lues — une supervision bruyante équivaut à une absence de supervision.

Le premier est calculé depuis le label `source:*` porté par chaque fiche. Sur les cinq anomalies documentées, trois l'ont été par les canaux internes — supervision pour ANO-2026-001 et 004, recette interne pour 002 —, une par le support client et une par constat fortuit : 60 %, à la limite de la cible.

2.4 Modalité des signalements

Sév.	Définition	Canal	Prise en charge	Correctif visé
S1	Service indisponible, perte ou fuite de données	E-mail + notification push	1 h ouvrée	4 h ouvrées
S2	Fonction clé inutilisable, sans contournement	E-mail	4 h ouvrées	2 j ouvrés
S3	Fonction dégradée, contournement possible	E-mail groupé quotidien	1 j ouvré	Sprint courant
S4	Défaut cosmétique	Backlog, sans notification	5 j ouvrés	Backlog

Escalade automatique. Une alerte S1 non acquittée sous 30 minutes est réémise ; une S2 ouverte depuis plus de 48 h est reclassée S1 — une anomalie majeure qui traîne finit par avoir le même effet qu'une panne. **Regroupement.** Les alertes S3 sont agrégées en un envoi quotidien : une alerte par occurrence sur un indicateur bruyant serait ignorée en quelques jours. Les politiques sont créées une par indicateur ; pour les indicateurs 4 et 7, la *log-based metric* est définie d'abord — ce qui n'est possible que parce que les logs sont émis en JSON (§ 2.2).

Confidentialité de la télémétrie

Les visuels d'une campagne sous embargo ne doivent jamais transiter par un service tiers. Trois garde-fous sont implémentés dans le code. **En-têtes filtrés** : `authorization` et `cookie` retirés de tout événement avant émission. **URL tronquées** : la chaîne de requête est supprimée côté front, car une URL signée de Supabase Storage donne un accès direct au fichier. **Identité minimale** : `sendDefaultPii: false`, seul l'identifiant technique est transmis — suffisant pour compter les utilisateurs touchés, insuffisant pour les identifier chez le sous-traitant. Les logs restent dans Cloud Logging (`europe-west1`), rétention 30 jours.

```

1  $ gcloud monitoring uptime list-configs --project myvisuals-back
2
3
4  | SONDE | CHEMIN | INTERVALLE | DELAI |
5  |-----|-----|-----|-----|
6  | Visuals.co API - aptitude (/health/ready) | /health/ready | 300s | 10s |
7
8
9  $ curl -s -o /dev/null -w '%{http_code}' https://<api>/health/ready
10 200
```

Figure 7 — La sonde externe : un uptime check interroge `/health/ready` toutes les 5 minutes depuis l'extérieur de l'infrastructure. C'est la seule sonde du dispositif qui ne soit pas hébergée par ce qu'elle surveille.

```

1  $ gcloud alpha monitoring policies list --project myvisuals-back
2
3  S2 · Indicateur 4 - Taux d'erreurs serveur (5xx > 2 % sur 5 min)
4  S1 · Indicateur 1 - Disponibilité de l'API (uptime check en échec)
5  S3 · Indicateur 5 - Latence p95 supérieure à 1,5 s sur 10 min
6  S2 · Indicateur 8 - Redémarrages de conteneur (plus de 3 par heure)
7
8  $ gcloud alpha monitoring channels list --project myvisuals-back
9
10  Astreinte Visuals.co - e-mail · email · deroy.flavien@gmail.com
```

Figure 8 — Les quatre politiques d'alerte actives et leur canal de notification. Chacune porte le numéro de l'indicateur du § 2.3 et sa sévérité, et documente la justification de son seuil dans le champ `documentation` de la politique.

ÉTAT DE MISE EN ŒUVRE AU MOMENT DE LA REMISE

Opérationnels — 8 indicateurs sur 12 : 1, 4, 5 et 8 par les politiques Cloud Monitoring ci-dessus ; 6 et 12 par Sentry ; 9 par les notifications GitHub Actions ; 10 par Dependabot. **Restent à configurer** : 2, 3, 7 et 11, qui supposent des *log-based metrics* dérivées des logs JSON et le suivi des quotas Supabase — seuils définis, donnée produite, règle de notification manquant. **Réserve** : le canal e-mail doit être vérifié par son destinataire pour émettre ; sans cela les politiques se déclenchent mais la notification n'est pas remise. Énoncer cet état plutôt que le masquer est ce qui rend le dispositif pilotable.

§ 3 — Collecte et consignation des anomalies

Compétence C4.2.1 — consigner les anomalies détectées en élaborant un **processus de collecte et consignation**, en utilisant des **outils de collecte** et en y intégrant **toutes les informations pertinentes**, afin de déterminer le correctif à mettre en place.

3.1 Le constat de départ : le coût mesuré de l'absence de processus

Le projet a d'abord fonctionné sans processus formel : les défauts étaient corrigés au fil de l'eau, et la seule trace subsistante était le message de commit. Cette pratique a un coût, mesuré sur ce projet même. **Trois commits successifs** — 14f394e , 9ee2d83 , a78a8bd — ont été nécessaires pour corriger un même problème de déploiement, faute d'avoir posé le diagnostic avant d'agir. Trois conséquences : la cause racine n'était pas écrite ; la récurrence n'était pas empêchée, rien ne garantissant qu'un correctif soit accompagné du test qui l'aurait détecté ; et rien n'était mesurable — sans consignation, impossible de savoir combien d'anomalies sont détectées par la supervision plutôt que subies par les utilisateurs.

Le processus tient en une règle : **toute anomalie confirmée donne lieu à une fiche, et tout correctif à un test qui échoue si le correctif est retiré.**

3.2 Les cinq canaux de détection

Par ordre décroissant de préférence : plus une anomalie est détectée tôt, moins elle coûte.

#	Canal	Outil	Délai typique	Qui la voit en premier
1	Intégration continue	GitHub Actions : lint, tests, smoke test, audit	minutes	Personne — bloquée avant fusion
2	Supervision automatique	Uptime check, Cloud Monitoring, Sentry	5–15 min	L'équipe
3	Recette interne	Tests manuels, revue de code	heures	L'équipe
4	Support client	Signalement utilisateur (formulaire [SUP] , e-mail)	variable	L'utilisateur
5	Constat fortuit	Aucun outil	indéterminé	Le hasard

Le canal 4 est coûteux : l'anomalie a déjà produit son effet et le diagnostic dépend du récit d'un tiers. Le canal 5 n'en est pas un — c'est l'aveu qu'aucun dispositif n'a fonctionné. Il a été ajouté après ANO-2026-005, restée invisible **deux jours** de toutes les sondes : le nommer permet de le mesurer, donc de le réduire (R2).

Les cinq anomalies du projet couvrent quatre canaux : 001 et 004 (supervision), 002 (recette interne), 003 (support client), 005 (constat fortuit).

3.3 Cycle de vie d'une anomalie

Le ticket ne se ferme pas au déploiement du correctif, mais à sa **vérification** : un correctif fusionné n'est qu'une hypothèse tant qu'il n'a pas été confronté à l'environnement réel — plusieurs anomalies de ce projet ne se manifestaient qu'en production.

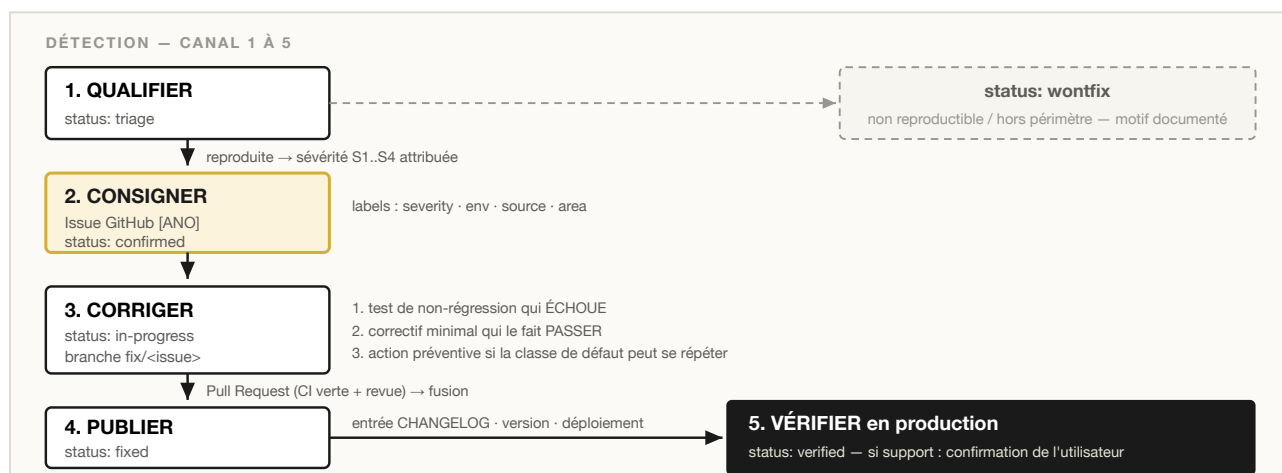


Figure 9 — Cycle de vie d'une anomalie : cinq états, transitions et sortie **wontfix** documentée.

3.4 Grille de sévérité et délais d'engagement

Sév.	Définition	Exemple vécu sur le projet	Prise en charge
S1	Service indisponible, perte ou fuite de données	ANO-2026-001 — API en échec au démarrage après déploiement	1 h ouvrée
S2	Fonction clé inutilisable, sans contournement	ANO-2026-003 — inscription Studio créant un compte Client	4 h ouvrées
S3	Fonction dégradée, contournement possible	ANO-2026-002 — saturation du limiteur sur le tableau de bord	1 j ouvré
S4	Défaut cosmétique, sans impact fonctionnel	Ruptures de mise en page sous 768 px	5 j ouvrés

La sévérité mesure l'impact utilisateur, jamais la difficulté technique. Une faute de frappe dans une condition d'autorisation est triviale à corriger et reste une S1 : c'est l'effet qui détermine la priorité, pas l'effort. ANO-2026-005 en est l'illustration extrême — une barre oblique de trop, classée S1.

3.5 Outils de collecte

Outil	Rôle	Ce qu'il apporte à la fiche
Formulaire GitHub [ANO]	Consignation	Champs obligatoires : sévérité, environnement, version déployée, étapes de reproduction, impact
Formulaire GitHub [SUP]	Réception des signalements	Verbatim utilisateur, contexte, engagement de réponse
Sentry	Détection + diagnostic	Pile d'appels, version concernée, nombre d'utilisateurs touchés, rejeu de session
Cloud Logging	Diagnostic	Ligne de log exacte, filtrable sur le <code>requestId</code>
X-Request-Id / INC-...	Corrélation	Relie signalement, log serveur et événement Sentry
Labels GitHub	Pilotage	<code>severity · env · source · area · status</code>
CHANGELOG.md	Traçabilité	Relie chaque correctif publié à sa version déployée (§ 7)

Les formulaires sont volontairement **bloquants** : les issues libres sont désactivées (`blank_issues_enabled: false`). Une fiche incomplète repart en demande d'information, ce qui coûte plus cher que de la remplir correctement.

La clé de corrélation est le mécanisme le plus déterminant du dispositif, et le moins visible. L'utilisateur signale « ça a planté, référence INC-260720-A4T2X » ; cette référence ouvre deux chemins : dans Sentry, la pile d'appels, la version et le rejeu de session ; dans Cloud Logging, la requête exacte, son statut et sa latence. Sans elle, le diagnostic part d'une reconstitution ; avec elle, une seule requête suffit.



Figure 10 — Configuration du formulaire [ANO] : liste **fermée** et champ `required: true`. La sévérité — comme l'environnement, le composant et le canal de détection, construits à l'identique — ne peut pas être saisie librement : c'est ce qui rend les indicateurs de pilotage du § 2.3 calculables. Le formulaire une fois rempli figure ci-après.

3.6 Contenu obligatoire d'une fiche

Une fiche exploitable répond à six questions. Si l'une reste sans réponse, la qualification n'est pas terminée.

Question	Pourquoi elle est obligatoire
Quoi — comportement observé vs attendu	Sans l'écart, il n'y a pas d'anomalie mais une insatisfaction
Où — environnement, composant, version déployée	Un correctif ne peut pas être validé si l'on ignore quelle version présentait le défaut
Quand — date, <code>requestId</code> , première occurrence	Détermine s'il s'agit d'une régression et identifie le déploiement en cause
Comment — étapes de reproduction	Une anomalie non reproductible ne peut pas être corrigée avec certitude, seulement contournée
Combien — utilisateurs touchés, contournement	Détermine la sévérité, donc la priorité
Preuves — logs, captures, lien Sentry	Évite de rejouer le diagnostic à chaque reprise du ticket

3.7 Les cinq anomalies consignées

Référence	Titre	Sév.	Canal	Issue
ANO-2026-001	Conteneur de production en échec au démarrage	S1	Supervision	#1
ANO-2026-002	Saturation du limiteur de débit	S3	Recette interne	#6
ANO-2026-003	Inscription Studio créant un compte Client	S2	Support client	#7
ANO-2026-004	Déploiements n'atteignant pas le service de production	S1	Supervision	#2
ANO-2026-005	Front bloqué par la politique CORS après un correctif	S1	Constat fortuit	#8

Chaque fiche existe sous deux formes : un document versionné dans le dépôt (`docs/anomalies/`) et une issue GitHub renseignée via le formulaire, labellisée et close après vérification en production. La première sert la traçabilité au fil du code, la seconde le pilotage — c'est d'elle que viennent les indicateurs du § 2.3.

Limites du processus. Une seule personne qualifie et corrige, sans regard extérieur sur la sévérité attribuée ; les échanges avec les utilisateurs vivent hors du dépôt, seul leur résumé y est consigné (R4) ; les indicateurs de pilotage sont relevés manuellement. Ces limites sont reprises et chiffrées au § 8.

§ 4 — Fiche de consignation ANO-2026-001

Fiche réelle, telle que consignée : *Conteneur de production en échec au démarrage après déploiement*. Cette page démontre la capacité à **consigner** ; le diagnostic, le correctif et l'action préventive font l'objet du § 5.

Référence	ANO-2026-001 · issue #1	Version affectée	1.4.0 (déploiement du 2026-07-21)
Sévérité	S1 — service indisponible	Version corrigée	1.5.0
Statut	Vérifiée — fermée	Détectée le	2026-07-21, ~21 h 10
Canal de détection	Supervision — échec du health check Cloud Run	Corrigée le	2026-07-21, 21 h 25 (a942974)
Environnement	Production — Cloud Run europe-west1	Vérifiée le	2026-07-21, 21 h 40
Composant	Chaîne CI/CD — image Docker de production	Labels	bug · severity:S1 · env:prod · source:monitoring · area:cicd · status:verified

Description

À la suite d'un déploiement sur Cloud Run, le service ne démarre plus : le conteneur s'arrête immédiatement après son lancement et Cloud Run le relance en boucle. Aucune route de l'API n'est joignable ; le front reste affiché mais toutes les requêtes échouent. Point notable : **toute la chaîne d'intégration continue était verte**.

Attendu / observé. Le conteneur devait démarrer et `/health` répondre `200 OK` ; il s'arrête au démarrage et redémarre en boucle. L'API devait servir `/api/**` ; toutes les requêtes échouent en 503. Un build Docker réussi devait impliquer une image exécutable ; le build réussit et l'image ne démarre pas.

Étapes de reproduction

Déterministe : se placer sur le commit précédant `a942974` ; `docker build -t visuals-api:repro ./server` → le build réussit ; `docker run -p 8080:8080 visuals-api:repro` → arrêt immédiat du process.

Preuves — journal de démarrage du conteneur

```
Error: Cannot find module '../utils/pagination'
Require stack:
- /app/controllers/auditLogController.js → /app/routes/auditLogRoutes.js
- /app/app.js → /app/server.js
code: 'MODULE_NOT_FOUND'
```

Impact

Utilisateurs touchés : **tous**, studios et clients. Durée : environ **15 minutes d'indisponibilité totale**. Contournement : aucun côté utilisateur.

The screenshot shows a GitHub issue page for the repository 'flavienderoy/myvisuals-back'. The issue title is '[ANO] Conteneur de production en échec au démarrage après déploiement #1'. It is marked as 'Closed'. The issue was opened 2 days ago by the owner 'flavienderoy'. The severity is 'S1 — Critique : service indisponible, perte ou fuite de données'. The environment is 'Production (Cloud Run + Vercel)'. The component is 'API — visuals-api (backend)'. The issue has six labels: 'area:cicd', 'bug', 'env:prod', 'severity:S1', 'source:monitoring', and 'status:verified'. There are no assignees, projects, or milestones.

Figure 11 — La fiche telle qu'elle existe dans GitHub Issues : champs structurés imposés par le formulaire, six labels de pilotage, statut fermé après vérification en production.

§ 5 — Traitement d'une anomalie détectée

Compétence C4.2.2 — traiter les anomalies détectées.

Détection

Le health check Cloud Run échoue après déploiement, alors que toute la chaîne d'intégration était verte. La trace désigne sans ambiguïté un module absent du système de fichiers de l'image, et non une erreur applicative.

Diagnostic

Trois hypothèses ont été examinées. Les hypothèses écartées font partie du raisonnement et sont conservées dans la fiche : elles évitent de refaire le même chemin lors d'une anomalie voisine.

Hypothèse	Vérification	Conclusion
Dépendance npm manquante	<code>utils/pagination</code> est un module <i>local</i> , pas un paquet npm	écartée
Fichier absent du dépôt	<code>git ls-files server/utils/</code> liste bien <code>pagination.js</code>	écartée
Fichier absent de l'image Docker	<code>docker run --rm visualstudio-repro ls /app</code> → pas de dossier <code>utils/</code>	confirmée

Le dossier existe dans le dépôt mais pas dans l'image : le défaut est dans les instructions de construction, pas dans le code applicatif.

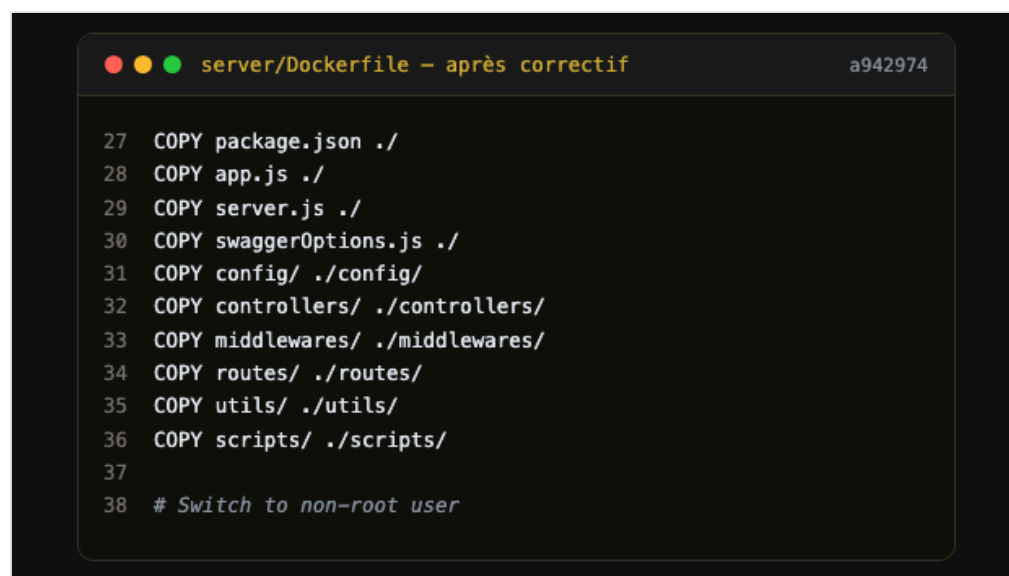
Cause racine, à deux niveaux

D'abord, le `Dockerfile` de production copiait les sources dossier par dossier, par énumération explicite. Cette liste avait été écrite quand `utils/` n'existait pas encore ; le dossier a été introduit ensuite — pagination, traitement d'images, journal d'activité, utilisé par plus de dix contrôleurs — ainsi que `swaggerOptions.js`, sans mise à jour de la liste. **Toute énumération manuelle de fichiers dérive à mesure que le projet grandit**, et rien dans la chaîne ne signalait cette dérive.

Ensuite, et c'est le facteur le plus important : **docker build n'exécute jamais l'application**. Il empile des couches de système de fichiers ; une image à laquelle il manque un fichier requis au démarrage se construit sans la moindre erreur. Le job `docker` vérifiait que l'image *se construit*, ce qui ne dit rien de sa capacité à *démarrer*. Le premier moment de vérité était la production — c'est-à-dire trop tard.

Correctif

Commit `a942974` : deux instructions `COPY` ajoutées. Il est volontairement **minimal** — rétablir le contenu de l'image, sans re-fonte du `Dockerfile` dans le même changement. Une restructuration de la stratégie de copie aurait rendu le retour arrière plus risqué au moment précis où la production était à l'arrêt.



```
27 COPY package.json ./
28 COPY app.js ./
29 COPY server.js ./
30 COPY swaggerOptions.js ./
31 COPY config/ ./config/
32 COPY controllers/ ./controllers/
33 COPY middlewares/ ./middlewares/
34 COPY routes/ ./routes/
35 COPY utils/ ./utils/
36 COPY scripts/ ./scripts/
37
38 # Switch to non-root user
```

Figure 12 — Le `Dockerfile` après correctif. Les lignes 30 et 35 — `swaggerOptions.js` et `utils/` — sont celles qu'ajoute `a942974` : elles manquaient à cette énumération, alors que les dossiers existaient dans le dépôt.

Pourquoi aucun test unitaire ne pouvait l'attraper

Les tests s'exécutent sur l'arborescence du dépôt, où `utils/` est présent : le défaut n'existe que dans l'artefact déployé. Le test de non-régression devait donc porter sur l'image — c'est l'action préventive qui en tient lieu.

Action préventive

Un smoke test a été ajouté au job `docker` : il ne construit plus seulement l'image, il **démarre réellement le conteneur** et interroge `/health` pendant 30 secondes. Portée de la mesure : elle ne corrige pas un fichier oublié, elle rend **structurellement impossible** le déploiement d'une image qui ne démarre pas — fichier manquant, erreur au chargement, variable d'environnement obligatoire absente, version de Node incompatible. Une classe entière de défauts passe de la production à la chaîne d'intégration.

C'est aussi ce qui a motivé `/health/ready` (§ 2.2) : le smoke test valide que le process démarre, la sonde d'aptitude qu'il peut effectivement servir.

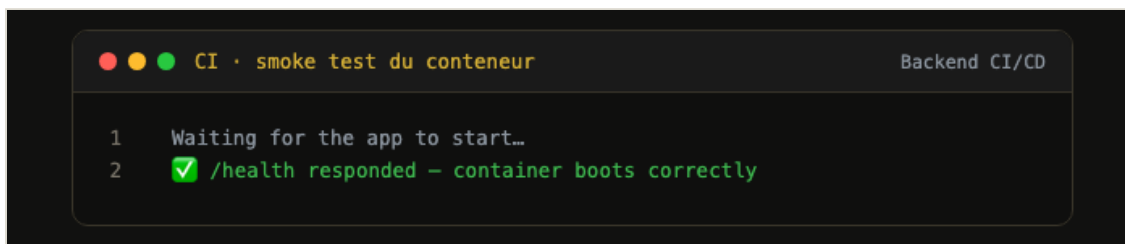


Figure 13 — Smoke test en intégration continue : le conteneur démarre et `/health` répond.

Contre-épreuve du mécanisme

Montrer qu'un garde-fou laisse passer une image saine ne prouve rien : encore faut-il vérifier qu'il **rejette** une image défectueuse. Le mécanisme de défaillance a donc été rejoué en reconstituant l'arborescence exacte que produisaient les `COPY` fautifs :

```
$ cd "$REPRO" && NODE_ENV=production node server.js
Error: Cannot find module '../utils/logger'
code: 'MODULE_NOT_FOUND'
```

Le process s'arrête au chargement, **avant toute écoute sur le port** : `/health` ne peut structurellement pas répondre, et le smoke test échoue. Le module cité diffère de l'incident d'origine parce que l'arborescence actuelle charge `config/monitoring.js` plus tôt — la classe de défaillance est identique.

Analyse post-incident

Question	Réponse
Pourquoi le défaut a-t-il été introduit ?	Le <code>Dockerfile</code> énumérait les dossiers un par un : un doublon de l'arborescence qu'aucun mécanisme ne tenait à jour
Pourquoi n'a-t-il pas été détecté plus tôt ?	La chaîne vérifiait que l'image se construit, jamais qu'elle démarre ; les tests s'exécutent sur le dépôt et non sur l'image
Qu'est-ce qui empêchera sa récurrence ?	Le smoke test qui démarre le conteneur et interroge <code>/health</code> à chaque exécution de la chaîne

ENSEIGNEMENT RETENU — ET SA GÉNÉRALISATION

Un artefact doit être testé sous la forme exacte dans laquelle il sera déployé. Tester le code source ne dit rien de l'image ; construire l'image ne dit rien de son exécution.

Ce projet a rencontré la même erreur à trois altitudes successives. ANO-2026-001 : un `docker build` réussi ne prouve pas qu'une image démarre. ANO-2026-004 : un `gcloud run deploy` réussi ne prouve pas qu'un service sert les utilisateurs — la chaîne déployait vers un service que personne n'interrogeait, pendant un mois. ANO-2026-005 : un service qui répond n'est pas une application utilisable — une barre oblique dans une variable d'environnement a bloqué tout le front pendant deux jours, sans qu'aucune sonde ne s'en aperçoive.

Les trois sont la même faute : **vérifier ce qui est commode à mesurer plutôt que ce qui compte réellement.** C'est ce constat qui fonde les recommandations R1 et R2 du § 8.

Amélioration identifiée, non retenue à ce stade. Remplacer l'énumération par `COPY . ./` avec un `.dockerignore` exhaustif supprimerait la cause première — écarté du correctif d'urgence pour ne pas élargir la surface de changement pendant une indisponibilité, porté en recommandation R3.

§ 6 — Problème résolu avec le support client

ANO-2026-003 — un utilisateur s'inscrit en tant que Studio et se retrouve dans l'espace Client. S2, signalée le 2026-07-23 à 22 h 05 par e-mail, corrigée le même soir à 23 h 30 (d20e4d3 , be52bf5 , faa9a27), publiée en 1.5.1, vérifiée le 2026-07-24 par confirmation de l'utilisateur.

SIGNALEMENT INITIAL — VERBATIM

« Bonjour, je viens de créer mon compte en choisissant **Studio** au moment de l'inscription, mais après la connexion je me retrouve dans l'espace client : je n'ai pas accès à mes projets ni au tableau de bord. J'ai réessayé avec une autre adresse mail, même résultat. »

Le verbatim est conservé sans reformulation : la reformulation prématurée est une source classique de fausse piste. **Accusé de réception à 22 h 20**, soit 15 minutes après le signalement — l'engagement S2 est de 4 h ouvrées (§ 2.4), il est tenu avec une large marge.

Qualification avec le demandeur

Trois éléments manquaient au signalement initial. C'est cette phase — un échange, non une investigation solitaire — qui a permis de cibler le diagnostic.

Question posée	Réponse obtenue	Ce que cela apporte
Le sélecteur Studio était-il bien actif à la soumission ?	Oui, capture d'écran du formulaire fournie	Écarte une erreur de saisie
Un message d'erreur est-il apparu ?	Aucun — l'inscription s'annonçait réussie	Le défaut est silencieux : rien ne remonte à l'utilisateur
Horodatage précis de l'inscription	2026-07-23, 21 h 58	Permet de cibler les logs serveur

Le demandeur a également accepté que son compte serve de cas de reproduction. **Reproduction obtenue en interne dans les 15 minutes**, sur un compte neuf : le défaut est systématique.

Diagnostic

La ligne de log relevée à l'horodatage communiqué désigne une violation de contrainte, non une erreur applicative : `new row for relation "profiles" violates check constraint "profiles_role_check"`. La contrainte en vigueur était `CHECK (role IN ('client', 'admin', 'member', 'owner'))` — **'studio' n'y figurait pas**.

Les deux écritures du chemin d'inscription échouaient donc : le trigger `handle_new_user`, avec son repli `COALESCE(..., 'client')`, aboutissait à un profil Client ; l'`upsert` du contrôleur échouait dans un `try/catch` qui journalisait sans interrompre le flux. L'inscription s'achevait « avec succès », sur un profil au mauvais rôle. Cause racine : **le vocabulaire des rôles avait divergé entre l'application et la base**.

Correctif

Migration `013_fix_profiles_role_check.sql` : contrainte alignée sur le vocabulaire applicatif et trigger rendu idempotent (`ON CONFLICT (id) DO UPDATE`). Côté API, l'erreur d'`upsert` cesse d'être avalée : elle est récupérée, journalisée, donc visible dans les logs et remontée à Sentry. Six cas de non-régression ont été ajoutés, vérifiés *par mutation* : le correctif retiré fait échouer 2 puis 3 tests.

Validation par l'utilisateur

Migration appliquée, 1.5.1 déployée, inscription Studio de test concluante en interne. Point facile à oublier : **le rôle du compte du demandeur a été corrigé en base** — un correctif ne répare pas rétroactivement les données déjà écrites, et sans cette étape l'utilisateur à l'origine du signalement restait bloqué. Retour au demandeur avec la version et la nature du défaut, puis **confirmation de sa part le 2026-07-24**. Le ticket n'a été clos qu'à cette étape : un correctif déployé n'est pas une résolution tant que le demandeur ne l'a pas constaté — c'est lui qui détient le critère de succès.

Limites assumées

Le support n'est aujourd'hui qu'une boîte e-mail et un formulaire [SUP] : les échanges vivent hors du dépôt et le diagnostic dépend du récit de l'utilisateur. Le bouton « Signaler un problème », attachant automatiquement le contexte de diagnostic (X-Request-Id, version, navigateur, rôle), est porté en recommandation **R4**. Par ailleurs, la contrainte `CHECK` n'est toujours pas testée : les tests s'exécutent contre un client Supabase simulé — précisément la frontière où le défaut est né. Le test d'intégration sur base réelle relève de **R1**, qui en fournit l'environnement.

§ 7 — Journal de version

Compétence C4.3.2 — établir un **journal des versions déployées** en y intégrant la **documentation des correctifs réalisés** pour suivre les différentes évolutions réalisées sur le logiciel.

Convention

Le journal suit *Keep a Changelog* 1.1.0 et le versionnage sémantique `MAJEUR.MINEUR.CORRECTIF`, avec les rubriques *Ajouté*, *Modifié*, *Corrigé*, *Supprimé* et *Sécurité*. Ce choix n'est pas cosmétique : l'historique du projet utilisait déjà des commits conventionnels (`feat:`, `fix:`), ce qui rend le journal **dérivable de l'historique réel** plutôt que rédigé à côté de lui. Le front est versionné séparément dans son propre dépôt : les deux applications ont des chaînes de déploiement distinctes — Cloud Run d'un côté, Vercel de l'autre — et les versions qui doivent être déployées ensemble sont signalées explicitement.

Chaîne de traçabilité

Elle est vérifiable de bout en bout, chaque maillon pouvant être contrôlé indépendamment.

Cinq maillons, chacun contrôlable indépendamment : (1) `CHANGELOG.md` atteste ce qui a changé, pourquoi et quelle anomalie est corrigée ; (2) le tag git `v1.6.2`, l'état exact du code ; (3) la Release GitHub, la publication datée et lisible sans cloner le dépôt ; (4) la révision Cloud Run, l'artefact effectivement déployé et la cible du retour arrière ; (5) `GET /health`, la version **réellement en service**, lue depuis `package.json`.

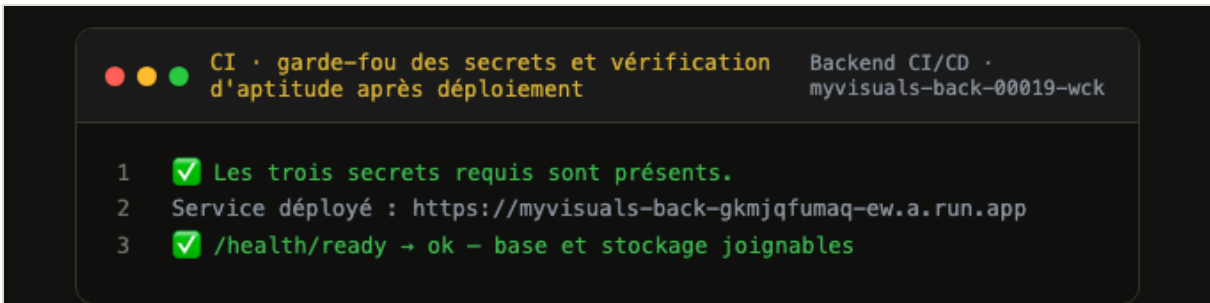


Figure 14 — Le maillon 4 en action : la chaîne vérifie l'aptitude du service sur l'URL réellement déployée avant de déclarer le déploiement réussi, et nomme la révision Cloud Run (`myvisuals-back-00019-wck`) à laquelle la version correspond.

Le garde-fou anti-dérive

La version était auparavant codée en dur dans `/health`, qui annonçait `1.0.0` — valeur figée depuis la première publication et divergente de la réalité déployée depuis six versions. Elle est désormais lue depuis `package.json`, un test de non-régression verrouille cette propriété, et `scripts/check-changelog.cjs` vérifie à chaque exécution de la chaîne que `package.json` et `CHANGELOG.md` annoncent la même version et que les versions y sont ordonnées — contrôle **bloquant avant déploiement** (figure 2). Pourquoi cela compte : **un journal qui ne correspond pas à la production ne trace rien**. Il donne une réponse fausse à la seule question qui compte en incident : quelle version tourne ?

Documentation des correctifs

Chaque correctif publié référence son anomalie, le ou les commits qui le portent, et la version qui le livre. Cette règle rend le journal utilisable dans les deux sens : d'une version vers les anomalies qu'elle corrige, et d'une anomalie vers la version qui la corrige. Sur l'ensemble du projet : **12 versions publiées, 12 tags et 12 Releases côté API ; 10 de chacun côté front.**

Version	Date	Anomalie corrigée	Commit(s)
1.6.3	2026-08-17	Récidive d'ANO-2026-005 — SENTRY_DSN effacée par le déploiement déclaratif	34145a0
1.6.2	2026-08-17	ANO-2026-005 (S1)	3e04a6b — normalisation CORS
1.6.1	2026-08-15	ANO-2026-004 (S1)	f85ecf0 — cible de déploiement, contrôle des secrets
1.5.2	2026-07-24	Chaîne de déploiement Cloud Run	14f394e, 9ee2d83, a78a8bd
1.5.1	2026-07-23	ANO-2026-003 (S2)	d20e4d3, be52bf5, faa9a27
1.5.0	2026-07-22	ANO-2026-001 (S1)	a942974
1.4.0	2026-07-20	ANO-2026-002 (S3)	2497a8f

Exemplaire du journal — entrée 1.6.2, reproduite intégralement

Extrait réel de `CHANGELOG.md`, sans coupe ni reformulation. Elle documente le correctif d'ANO-2026-005.

[1.6.2] — 2026-08-17

Corrigé

- **ANO-2026-005 — Front-end intégralement bloqué par la politique CORS** (S1, exposition $\approx$ 2 jours). `CLIENT_URL` portait une barre oblique finale, que l'en-tête `Origin` d'un navigateur ne comporte jamais : la comparaison exacte échouait, le préflight répondait 204 sans `Access-Control-Allow-Origin`, et toutes les requêtes du front étaient rejetées. La valeur fautive provenait du secret GitHub, appliqué pour la première fois par le correctif d'ANO-2026-004 : **une régression introduite par un correctif**.
- **Origines CORS normalisées des deux côtés de la comparaison** (slash terminal, casse, espaces). Une erreur de saisie dans une variable d'environnement ne peut plus rendre l'API inutilisable.
- **Secret `CLIENT_URL` corrigé** en amont, sans quoi le déploiement suivant aurait réintroduit le défaut.

Sécurité

- **Rejets CORS journalisés** en `WARNING` avec l'origine refusée. Un rejet est soit une tentative d'accès illégitime, soit une erreur de configuration : sans trace, les deux étaient indiscernables et silencieux.

Tests

- 6 cas de non-régression (`__tests__/cors.test.js`), dont le scénario exact du défaut — `CLIENT_URL` renseignée avec le slash. Vérifiés par mutation : la normalisation retirée, 2 tests échouent. Suite portée à **96 tests**.

Quatre propriétés de cette entrée méritent d'être relevées, parce qu'elles valent pour toutes les autres. Elle **nomme l'anomalie corrigée et sa sévérité**, ce qui relie le journal aux fiches du § 3. Elle **énonce la durée d'exposition** plutôt que de la taire. Elle **documente les correctifs de la chaîne de livraison**, et pas seulement du code applicatif — c'est là que se situaient trois des cinq anomalies du projet. Et elle **assume qu'une régression a été introduite par un correctif antérieur**, en le nommant : un journal qui ne consigne que les succès perd sa valeur d'outil de diagnostic.

La rubrique *Sécurité* de la 1.6.0 suit la même règle : les 14 vulnérabilités corrigées y sont nommées paquet par paquet, avec le résultat vérifiable `npm audit` → 0 vulnérabilité et le garde-fou permanent qui l'accompagne. Les entrées non publiées sont conservées sous un en-tête *Non publié*, aujourd'hui vide.

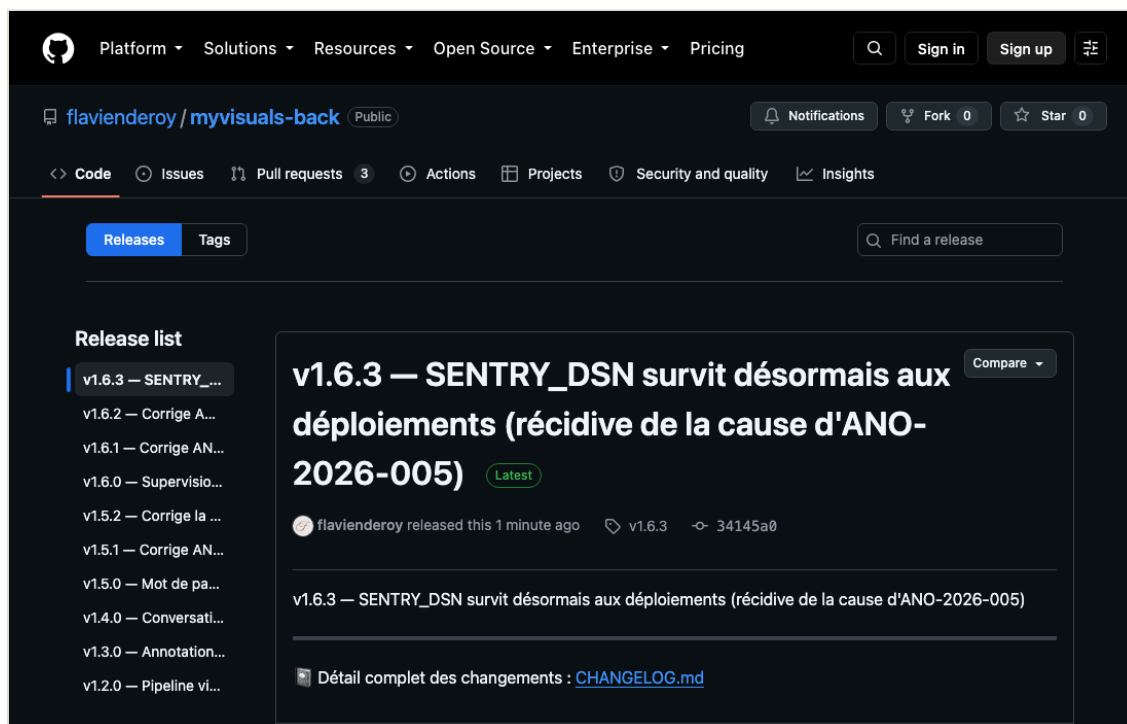


Figure 15 — Les Releases GitHub publiées : chaque version porte son numéro, sa date, son commit et la mention de l'anomalie qu'elle corrige — lisibles sans cloner le dépôt.

§ 8 — Recommandations argumentées d'amélioration

Chaque recommandation part d'une limite réellement rencontrée et documentée, non d'un principe.

Réf.	Constat → solution	Effort	Bénéfice mesurable	Priorité
R1	Certaines régressions ne sont détectables qu'en production. faute d'environnement de recette → service <code>visuals-api-staging</code> sur un projet Supabase dédié, même chaîne de déploiement. Fournit aussi l'environnement des tests d'intégration sur base réelle qui manque à ANO-2026-003.	2 j	Part des anomalies détectées en production ramenée sous 20 % ; montées majeures validées hors production	Haute
R2	Les sondes sont synthétiques : aucune ne se comporte comme un navigateur → scénario Playwright exécuté périodiquement contre la production (se connecter, charger le tableau de bord, vérifier qu'un projet s'affiche).	3 j	Couvre les régressions fonctionnelles silencieuses ; ANO-2026-005 aurait été détectée en 15 min au lieu de 2 jours	Haute
R3	L'énumération des <code>COPY</code> du <code>Dockerfile</code> dérive à chaque nouveau dossier → <code>COPY . ./</code> avec un <code>.dockerignore</code> exhaustif.	2 h	Supprime la cause première d'ANO-2026-001 : la classe de défaut disparaît au lieu d'être détectée	Moyenne
R4	Le diagnostic d'un signalement dépend du récit de l'utilisateur → bouton « Signaler un problème » attachant automatiquement <code>X-Request-Id</code> , version, navigateur et rôle.	3 j	Supprime la phase de qualification (15 min à plusieurs heures sur ANO-2026-003) ; fiches complètes dès la réception	Moyenne
R5	<code>npm audit</code> ne couvre pas les dépendances système de l'image Alpine → analyse d'image (Trivy) en intégration continue et génération d'un SBOM CycloneDX à chaque publication.	1 j	Étend l'indicateur « 0 vulnérabilité élevée » aux couches système ; réponse immédiate à « suis-je exposé à cette CVE ? »	Moyenne
R6	Quatre indicateurs (2, 3, 7, 11) produisent la donnée sans émettre de signalement → créer les <i>log-based metrics</i> correspondantes et leurs politiques de seuil, et vérifier le canal e-mail.	1 h	Porte la couverture d'alerte de 8 à 12 indicateurs sur 12 ; délai de détection mesurable	Haute
R7	La temporisation exponentielle côté front n'est pas testée (lacune reconnue d'ANO-2026-002), et le projet n'a pas d'astreinte → test automatisé avec minuteurs simulés, puis rotation d'astreinte dès qu'une seconde personne rejoint le projet.	1 j	Verrouille le correctif d'ANO-2026-002 ; réduit le délai de prise en charge d'une S1 hors heures ouvrées	Basse

R1 et R2 sont prioritaires parce qu'elles ferment les deux angles morts qui ont réellement produit des incidents : l'absence d'environnement où éprouver un changement avant la production, et l'absence de mesure du service tel que l'utilisateur en fait l'expérience. R3 est la seule qui *supprime* une cause plutôt que d'ajouter une détection — c'est la forme de correction la plus solide, et la moins fréquente.

Limites qui subsistent

Elles sont énoncées parce qu'un dispositif dont on connaît les angles morts est plus sûr qu'un dispositif supposé complet.

- **Pas d'astreinte hors heures ouvrées** — une panne S1 nocturne sera détectée par la sonde externe mais traitée le lendemain. C'est un choix assumé, cohérent avec les délais annoncés au § 2.4 plutôt qu'avec un engagement affiché et non tenu.
- **Couverture d'alerte partielle** — 8 indicateurs sur 12 émettent un signalement ; les 4 restants attendent leurs *log-based metrics* (R6). Le canal e-mail reste à vérifier par son destinataire.
- **Projet porté par une seule personne** — pas de regard extérieur sur la qualification d'une anomalie, donc un risque de sous-évaluer une sévérité ; les indicateurs de pilotage sont relevés manuellement. S'y ajoutent les quotas des offres gratuites (Sentry, UptimeRobot) et l'absence de budget d'alerte Google Cloud contre une facture imprévue.
- **Couverture de test à la frontière de la base** — les tests s'exécutent contre un client Supabase simulé : la contrainte `CHECK` d'ANO-2026-003 n'est pas testée, précisément là où le défaut est né. Relève de R1.

BILAN

Le dispositif est passé d'aucun suivi — ni journal de version cohérent, ni consignation, ni audit de dépendances, ni sonde autre qu'un `/health` qui répondait `ok` sans rien vérifier — à un ensemble outillé : 5 anomalies documentées et vérifiées, 12 versions tracées de bout en bout, 171 tests dont 19 dédiés aux sondes et à la politique CORS, 33 vulnérabilités corrigées, et cinq contrôles bloquants avant tout déploiement (journal, audit, smoke test, secrets, aptitude).

Deux des anomalies découvertes étaient antérieures et invisibles : la production servait du code vieux d'un mois sans que rien ne l'indique. C'est la mesure la plus juste de ce que vaut un dispositif de supervision — non pas ce qu'il affiche quand tout va bien, mais ce qu'il révèle le jour où on l'installe.